ASSIGNMENT-12.5

2303A51600

BATCH-29

TASK-1:

PROMPT:

Generate a Python program that implements the Merge Sort algorithm.

Requirements:

1. Create a function merge_sort(arr) that sorts a list in ascending order.

2. Include a detailed docstring explaining:

  - Time Complexity

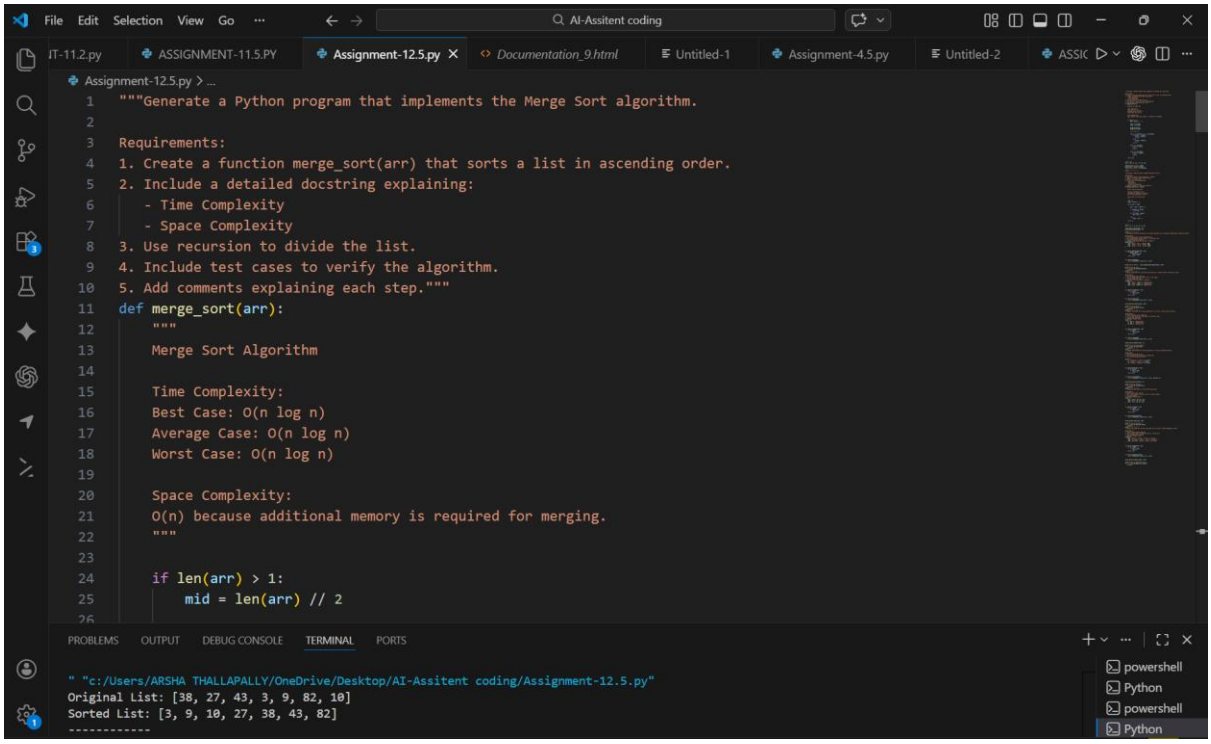
  - Space Complexity

3. Use recursion to divide the list.

4. Include test cases to verify the algorithm.

5. Add comments explaining each step.

CODE:



OBSERVATION:

Merge Sort correctly sorted the list in ascending order.
The algorithm divides the list into smaller parts and merges them after sorting.

TASK-2:

PROMPT:

Generate a Python program implementing Binary Search.

Requirements:

1. Create a function binary_search(arr, target)

2. Return the index if the element exists

3. Return -1 if not found

4. Include docstrings explaining:

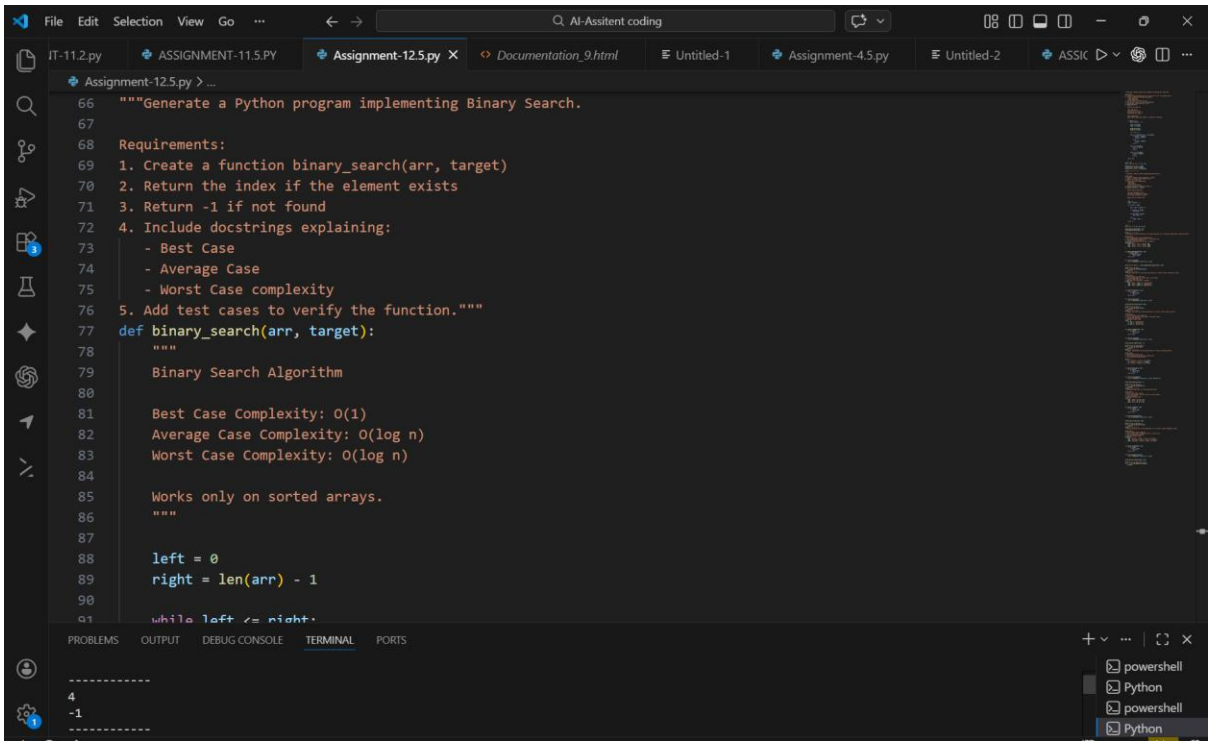
- Best Case

- Average Case

- Worst Case complexity

5. Add test cases to verify the function.

CODE:



OBSERVATION:

Binary Search efficiently finds elements in a sorted list by repeatedly dividing the search space in half.

TASK-3:

PROMPT:

Recommend suitable searching and sorting algorithms for a healthcare appointment scheduling system.

Requirements:

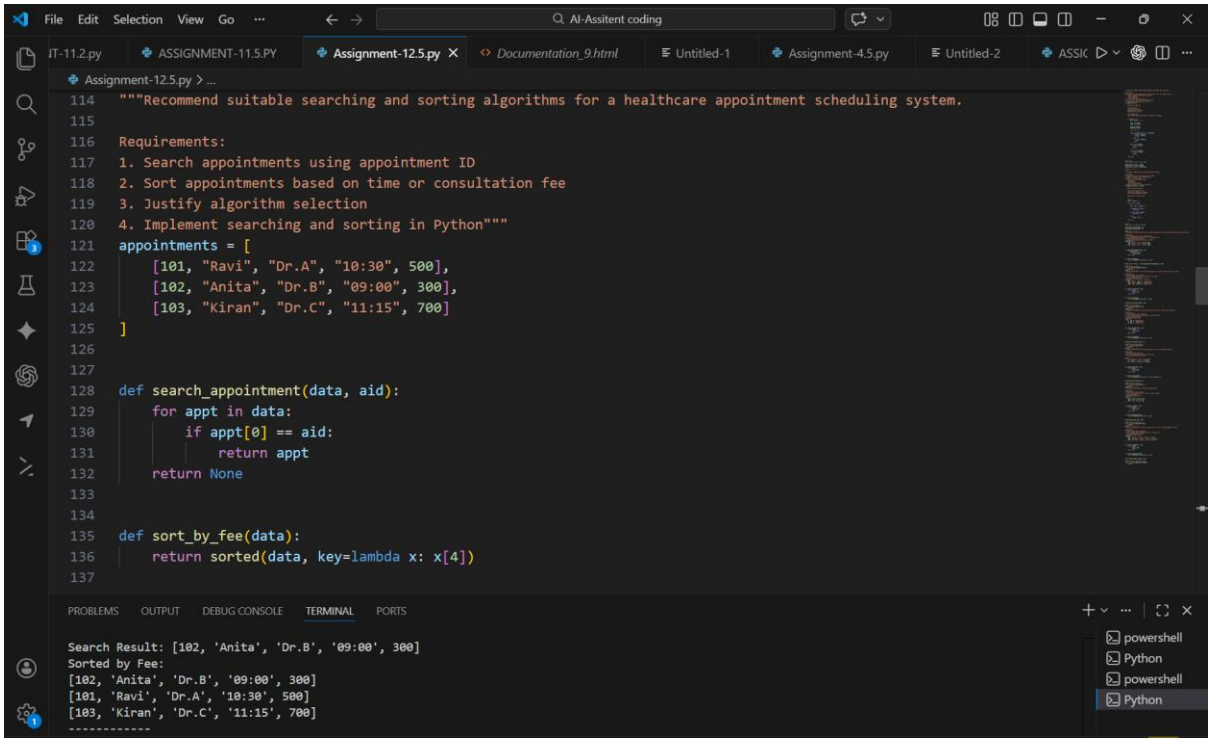
1. Search appointments using appointment ID

2. Sort appointments based on time or consultation fee

3. Justify algorithm selection

4. Implement searching and sorting in Python

CODE:



OBSERVATION:

The system successfully searched appointments by ID and sorted them based on consultation fees.

TASK-4:

PROMPT:

Design algorithms for searching and sorting in a railway ticket reservation system.

Requirements:

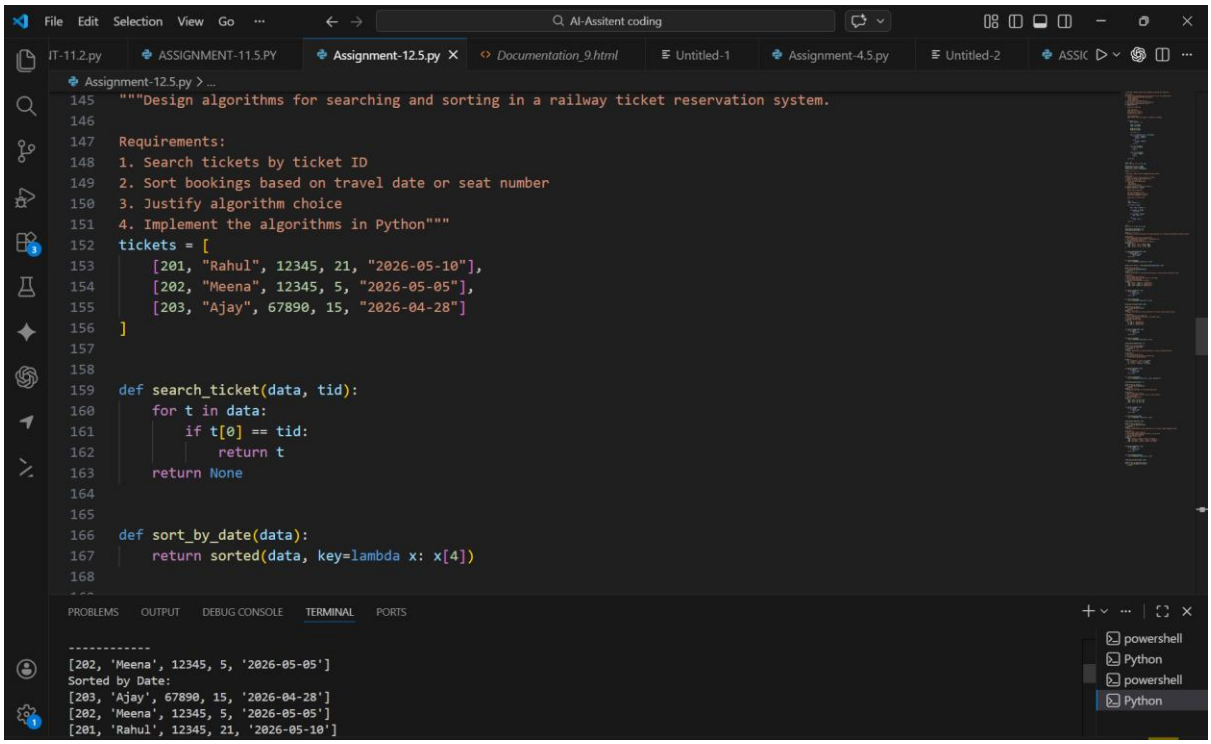
1. Search tickets by ticket ID

2. Sort bookings based on travel date or seat number

3. Justify algorithm choice

4. Implement the algorithms in Python

CODE:



OBSERVATION: The program successfully searches ticket records using the **ticket ID** and returns the correct passenger details. The sorting function correctly organizes the bookings based on **travel date**, helping manage reservations efficiently.

TASK-5:

PROMPT:

Suggest searching and sorting algorithms for a hostel room allocation system.

Requirements:

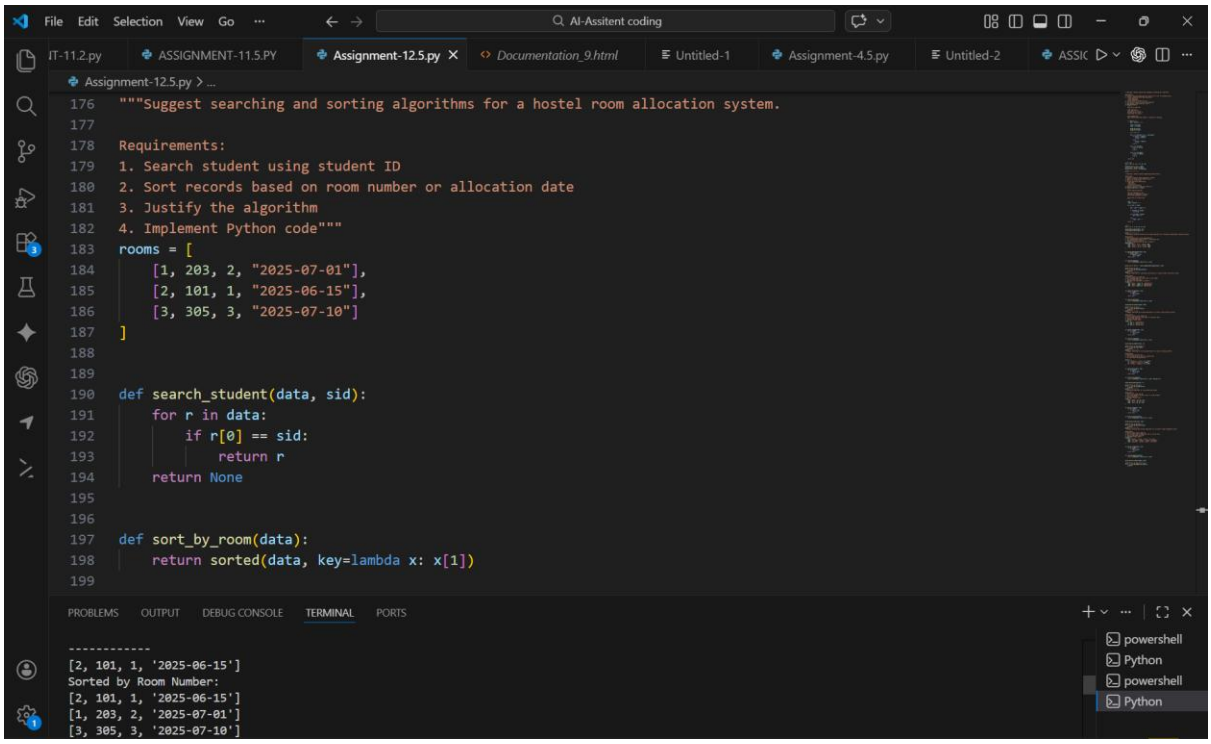
1. Search student using student ID

2. Sort records based on room number or allocation date

3. Justify the algorithm

4. Implement Python code

CODE:



OBSERVATION:

The system correctly finds student allocation details using the **student ID**. The sorting function arranges records by **room number**, making hostel room management more organized and easier to access.

TASK-6:

PROMPT:

Suggest searching and sorting algorithms for a movie streaming platform.

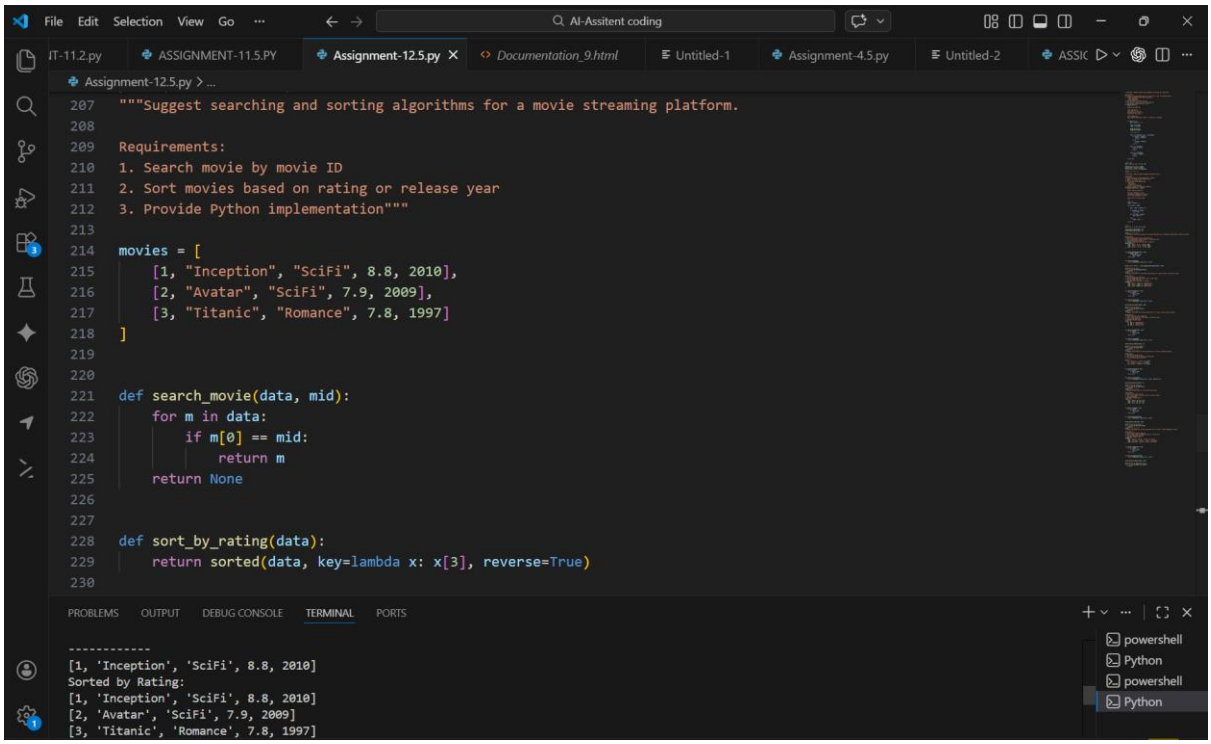
Requirements:

1. Search movie by movie ID

2. Sort movies based on rating or release year

3. Provide Python implementation

CODE:



OBSERVATION:

The program successfully searches movies using the **movie ID** and retrieves the correct movie information. The sorting function organizes movies based on **rating**, helping users view higher-rated movies first.

TASK-7:

PROMPT:

Recommend algorithms for crop monitoring system.

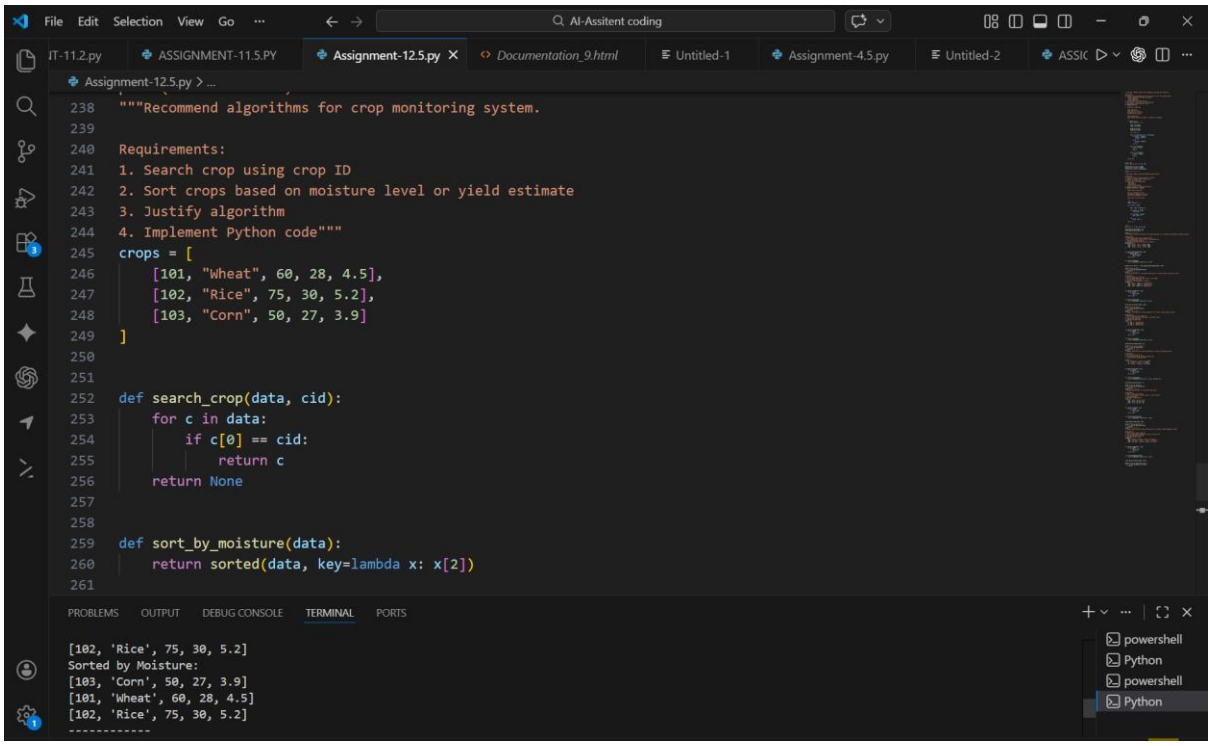
Requirements:

1. Search crop using crop ID

2. Sort crops based on moisture level or yield estimate

3. Justify algorithm

4. Implement Python codeCODE:



OBSERVATION:

The system accurately retrieves crop information using the **crop ID**. The sorting function arranges crop data by **soil moisture level**, which helps farmers monitor and analyze crop conditions efficiently.

TASK-8:

PROMPT:

Design searching and sorting algorithms for an airport flight management system.

Requirements:

1. Search flight using flight ID

2. Sort flights based on departure time or arrival time

3. Justify algorithm selection

4. Implement Python code

CODE:

```python

Requirements:
1. Search flight using flight ID
2. Sort flights based on departure time or arrival time
3. Justify algorithm selection
4. Implement Python code"""
flights = [
    [301, "Indigo", "10:30", "12:45", "On Time"],
    [302, "Air India", "08:15", "10:30", "Delayed"],
    [303, "SpiceJet", "14:00", "16:20", "On Time"]
]
def search_flight(data, fid):
    for f in data:
        if f[0] == fid:
            return f
    return None
def sort_by_departure(data):
    return sorted(data, key=lambda x: x[2])
print(search_flight(flights, 302))

print("Sorted by Departure Time:")
for f in sort_by_departure(flights):
    print(f)
```

```
------------
[302, 'Air India', '08:15', '10:30', 'Delayed']
Sorted by Departure Time:
[302, 'Air India', '08:15', '10:30', 'Delayed']
[301, 'Indigo', '10:30', '12:45', 'On Time']
[303, 'SpiceJet', '14:00', '16:20', 'On Time']
PS C:\Users\ARSHA THALLAPALLY\OneDrive\Desktop\AI-Assitent coding>
```

OBSERVATION:

The program correctly searches flight details using the **flight ID**. The sorting function organizes flights based on **departure time**, allowing better flight schedule management.